

1. The SQL script to initialize the tables and data for our project can be found under the scripts folder titled "updatedStockManager.sql". The path to the file from the project repository is: scripts/updatedStockManager.sql.
2. Our final project allows users to view, insert, update, and delete stocks, listing the relevant information of a given stock such as their price, volume, market cap, and the exchange they are listed on. Additionally, our project supports other relations such as relations of Stock Exchanges and Locations, and Stock Exchanges and their currency. With future additions, this would allow users to track and maintain a portfolio of stocks.
3. Our final schema differs from the schema we turned in only from an implementation perspective. Our final schema (and our SQL script) supports the same relations as the schema we previously turned in, however, we only implemented user functions for the relevant relations corresponding to the criteria. Chiefly, the main relations we focused on was the STOCKEXCHANGE relation (the normalized relation of Stock with Stock Exchange) and the LocationAndCurrency relation.
4. List of SQL queries and link to the query in the code. All queries are in project_d0o2e_p4q1k_p7b3b/appService.js, and the line numbers are included:
 - a. INSERT (206-239):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L206C1-L239C2

```
async function insertStockExchange(
  tickerID,
  price,
  marketCap,
  shares,
  volume,
  StockExchangeName
) {
  if (!sanitizeHelper(tickerID) || !sanitizeHelper(StockExchangeName)) {
    console.error("insertStockExchange blocked due to suspicious input");
    return false;
  }

  return await withOracleDB(async (connection) => {
    const checkValidForeignKey = await connection.execute(
      `SELECT COUNT(*) FROM STOCKEXCHANGE WHERE StockExchangeName = :StockExchangeName`,
      [StockExchangeName]
    );

    if (checkValidForeignKey.rows[0][0] === 0) {
      return "ForeignKeyViolated";
    }

    const result = await connection.execute(
      `INSERT INTO STOCKEXCHANGE (tickerID, price, marketCap, shares, volume, StockExchangeName) VALUES (:tickerID, :price, :marketCap, :shares, :volume, :StockExchangeName)`,
      [tickerID, price, marketCap, shares, volume, StockExchangeName],
      { autoCommit: true }
    );

    return result.rowsAffected && result.rowsAffected > 0;
  }).catch(() => {
    return false;
  });
}
```

- b. UPDATE (285-335):
https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b

```
async function updateStockExchange(
  tickerID,
  newPrice,
  newMarketCap,
  newVolume,
  newShares,
  newExchangeName
) {
  if (!sanitizeHelper(tickerID) || !sanitizeHelper(newExchangeName)) {
    console.error("updateStockExchange blocked due to suspicious input");
    return false;
  }

  return await withOracleDB(async (connection) => {
    const updates = [];
    const params = {};
    params.tickerID = tickerID;

    if (newPrice != null && newPrice !== "") {
      updates.push("price = :price");
      params.price = newPrice;
    }
    if (newMarketCap != null && newMarketCap !== "") {
      updates.push("marketCap = :marketCap");
      params.marketCap = newMarketCap;
    }
    if (newVolume != null && newVolume !== "") {
      updates.push("volume = :volume");
      params.volume = newVolume;
    }
    if (newShares != null && newShares !== "") {
      updates.push("shares = :shares");
      params.shares = newShares;
    }
    if (newExchangeName != null && newExchangeName !== "") {
      updates.push("StockExchangeName = :exchangeName");
      params.exchangeName = newExchangeName;
    }

    const result = await connection.execute(
      `
      UPDATE STOCKEXCHANGE
      SET ${updates.join(", ")}
      WHERE tickerID = :tickerID
      `,
      params,
      { autoCommit: true }
    );
    return result.rowsAffected && result.rowsAffected > 0;
  }).catch(() => false);
}
```

C2

- c. DELETE (410-426):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/6e119f2f905bb886f090e7b573886cc2fbe3b6cb/appService.js#L366C1-L377C

```

async function deleteLocationAndCurrency(CurrencyLocation) {
  return await withOracleDB(async (connection) => {
    const result = await connection.execute(
      `DELETE FROM LocationAndCurrency WHERE CurrencyLocation = :currencyLocation`,
      [CurrencyLocation],
      { autoCommit: true }
    );
    return result.rowsAffected && result.rowsAffected > 0;
  }).catch(() => {
    return false;
  });
}

```

- d. SELECTION (428-456):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L428C1-L456C2

```

async function selectStockWithConditions(conditions) {
  return await withOracleDB(async (connection) => {
    let baseQuery = "SELECT * FROM STOCKEXCHANGE";
    let clauseList = [];
    let bindParams = {};

    conditions.forEach((cond, i) => {
      const bindKey = `val${i}`;
      if (i === 0) {
        clauseList.push(`${cond.attribute} ${cond.operator} :${bindKey}`);
      } else {
        clauseList.push(
          `${cond.logic} ${cond.attribute} ${cond.operator} :${bindKey}`
        );
      }
      bindParams[bindKey] = cond.value;
    });

    if (clauseList.length > 0) {
      baseQuery += " WHERE " + clauseList.join(" ");
    }

    const result = await connection.execute(baseQuery, bindParams);
    return result.rows;
  }).catch((err) => {
    console.error("selectStockWithConditions error:", err);
    return false;
  });
}

```

- e. PROJECTION (612-637):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L612C1-L637C2

```

async function projectStockExchange(columns) {
  return await withOracleDB(async (connection) => {
    const allowedColumns = [
      "TICKERID",
      "PRICE",
      "MARKETCAP",
      "SHARES",
      "VOLUME",
      "STOCKEXCHANGENAME",
    ];
    const selected = columns.filter((col) =>
      allowedColumns.includes(col.toUpperCase())
    );

    if (selected.length === 0) {
      throw new Error("No valid columns selected");
    }

    const sql = `SELECT DISTINCT ${selected.join(", ")} FROM STOCKEXCHANGE`;
    const result = await connection.execute(sql);
    return result.rows;
  }).catch((err) => {
    console.error("projectStockExchange error:", err);
    return false;
  });
}

```

f. JOIN (469- 487):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L469C1-L488C1

```

// joins location currency and exchange location tables based on WHERE = location
async function joinLocationCurrencyExchange(currencyLocation) {
  return await withOracleDB(async (connection) => {
    const result = await connection.execute(
      "SELECT S.StockExchangeName, S.CurrencyLocation, L.Currency FROM StockExchangeLocation S JOIN LocationAndCurrency L ON S.CurrencyLocation = L.CurrencyLocation WHERE S.CurrencyLocation = :currencyLocation",
      [currencyLocation]
    );
    // console.log(currencyLocation);
    // console.log(1);
    // console.log(result);
    // console.log(2);
    // console.log(result.rows);

    return result.rows;
  }).catch(() => {
    console.log("failed");
    return [];
  });
}

```

g. Aggregation with GROUP BY (514-528):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L514C1-L528C2

```

async function aggregateGroupByStockExchange() {
  return await withOracleDB(async (connection) => {
    const result = await connection.execute(
      `SELECT se.StockExchangeName, SUM(se.MarketCap)
      FROM STOCKEXCHANGE se
      GROUP BY se.StockExchangeName
      ORDER BY se.StockExchangeName ASC
      `
    );
    return result.rows;
  }).catch((error) => {
    console.error("Error in aggregateGroupByStockExchange:", error);
    return [];
  });
}

```

```

'SELECT se.StockExchangeName, SUM(se.MarketCap)
FROM STOCKEXCHANGE se
GROUP BY se.StockExchangeName
ORDER BY se.StockExchangeName ASC'

```

- h. Aggregation with HAVING (530-545):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L530C1-L545C2

```

async function aggregateHavingStockExchange() {
  return await withOracleDB(async (connection) => {
    const result = await connection.execute(
      `SELECT se.StockExchangeName, SUM(se.Volume), AVG(se.Price)
      FROM STOCKEXCHANGE se
      GROUP BY se.StockExchangeName
      HAVING AVG(se.Price) > 15
      ORDER BY se.StockExchangeName ASC
      `
    );
    return result.rows;
  }).catch((error) => {
    console.error("Error in aggregateGrouByStockExchange:", error);
    return [];
  });
}

```

```

'SELECT se.StockExchangeName, SUM(se.Volume), AVG(se.Price)
FROM STOCKEXCHANGE se
GROUP BY se.StockExchangeName
HAVING AVG(se.Price) > 15
ORDER BY se.StockExchangeName ASC'

```

- i. Nested Aggregation with GROUP BY (565-581):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b

</blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L565C1-L581C2>

```
async function nestedAggregationStockExchange() {
  return await withOracleDB(async (connection) => {
    const result = await connection.execute(
      `SELECT se.StockExchangeName, AVG(se.Price) AS AvgPrice
      FROM STOCKEXCHANGE se
      GROUP BY se.StockExchangeName
      HAVING AVG(se.Price) <= ALL (SELECT AVG(se2.Price)
      FROM STOCKEXCHANGE se2
      GROUP BY se2.StockExchangeName)
      `
    );
    return result.rows;
  }).catch((error) => {
    console.error("Error in aggregateGrouByStockExchange:", error);
    return [];
  });
}
```

```
'SELECT se.StockExchangeName, AVG(se.Price) AS AvgPrice
  FROM STOCKEXCHANGE se
  GROUP BY se.StockExchangeName
  HAVING AVG(se.Price) <= ALL (SELECT AVG(se2.Price)
    FROM STOCKEXCHANGE se2
    GROUP BY se2.StockExchangeName)'
```

j. Division (489-512):

https://github.students.cs.ubc.ca/CPSC304-2024W-T2/project_d0o2e_p4q1k_p7b3b/blob/8c1e6457152a67d989a99b2ef3e13e35aac3eaef/appService.js#L489C1-L512

```

async function divideStockExchange() {
  return await withOracleDB(async (connection) => {
    const result = await connection.execute(
      `SELECT DISTINCT sel.StockExchangeName
      FROM StockExchangeLocation sel
      WHERE NOT EXISTS (
        SELECT column_value
        FROM TABLE(SYS.ODCIVARCHAR2LIST('AMZON', 'APPLE', 'NTFLX', 'GOOGL', 'FACEB'))
      MINUS
      (SELECT DISTINCT se.TickerID
      FROM StockExchange se
      WHERE se.StockExchangeName = sel.StockExchangeName
      AND (se.TickerID = 'AMZON' OR
        se.TickerID = 'APPLE' OR
        se.TickerID = 'NTFLX' OR
        se.TickerID = 'GOOGL' OR se.TickerID = 'FACEB'))
      )`
    );
    return result.rows;
  }).catch((error) => {
    console.error("Error in divideStockExchange:", error);
    return [];
  });
}

```

```

'SELECT DISTINCT sel.StockExchangeName
  FROM StockExchangeLocation sel
  WHERE NOT EXISTS (
    SELECT column_value
    FROM TABLE(SYS.ODCIVARCHAR2LIST('AMZON', 'APPLE', 'NTFLX',
'GOOGL', 'FACEB'))
  MINUS
  (SELECT DISTINCT se.TickerID
  FROM StockExchange se
  WHERE se.StockExchangeName = sel.StockExchangeName
  AND (se.TickerID = 'AMZON' OR
    se.TickerID = 'APPLE' OR
    se.TickerID = 'NTFLX' OR
    se.TickerID = 'GOOGL' OR se.TickerID = 'FACEB'))
  )'

```